

☒ Claude Code Master Prompt — Crypto Arbitrage Intelligence System

Paste this entire prompt into Claude Code (claude.ai/code) to scaffold the full system.

Context

You are the principal engineer building a **production-grade crypto arbitrage system** targeting funding-rate, spot-perp basis, and cross-exchange arbitrage strategies. The system is built on GCP, uses Terraform for infrastructure, and integrates AI agents for ops/monitoring only (not autonomous trading).

Stack:

- Language: Python 3.12 (services), TypeScript (dashboard API)
- Cloud: Google Cloud Platform (Cloud Run, Pub/Sub, BigQuery, Memorystore/Redis, Secret Manager, Cloud Monitoring)
- IaC: Terraform (modular, per-service)
- Exchange Library: CCXT Pro (WebSocket + REST)
- Execution Engine: Hummingbot (spot-perp, funding-rate strategies)
- AI Ops: MCP server exposing Hummingbot + exchange status to Claude/Gemini
- CI/CD: GitHub Actions (PR-based, rebase-before-merge)
- Dev Tools: Cursor, Warp terminal

Repository Structure to Generate

```
crypto-arb-system/  
├── README.md  
├── .github/  
│   ├── workflows/  
│   │   ├── ci.yml           # Lint, test, type-check on PR  
│   │   └── deploy.yml       # Cloud Run deployment on merge to main  
├── infra/  
│   └── terraform/  
│       ├── main.tf  
│       ├── variables.tf  
│       ├── outputs.tf  
│       └── modules/  
│           ├── cloud-run/    # Reusable Cloud Run module  
│           ├── pubsub/       # Topics and subscriptions  
│           ├── bigquery/     # Datasets and tables  
│           ├── memorystore/  # Redis for live spread cache  
│           └── secret-manager/ # Exchange API keys (NO withdraw perms)
```

```

├── monitoring/                # Alert policies, kill-switch triggers
├── environments/
│   ├── dev/
│   └── prod/
├── services/
│   ├── market-data/          # CCXT Pro WebSocket collectors
│   │   ├── Dockerfile
│   │   ├── main.py
│   │   ├── collectors/
│   │   │   ├── binance_us.py
│   │   │   ├── coinbase.py
│   │   │   ├── kraken.py
│   │   │   └── hyperliquid.py
│   │   ├── models.py          # Pydantic tick/orderbook models
│   │   └── requirements.txt
│   ├── funding-rate-service/  # Funding rate collector + normalizer
│   │   ├── Dockerfile
│   │   ├── main.py
│   │   ├── sources/
│   │   │   ├── coinglass.py   # CoinGlass API (read funding rates)
│   │   │   ├── arbitrage_scanner.py # ArbitrageScanner.io
│   │   │   └── ccxt_funding.py # CCXT native funding rate polling
│   │   ├── normalizer.py      # Normalize to standard FundingRate model
│   │   ├── models.py
│   │   └── requirements.txt
│   ├── opportunity-engine/    # Detect + score arbitrage opportunities
│   │   ├── Dockerfile
│   │   ├── main.py
│   │   ├── strategies/
│   │   │   ├── base.py        # Abstract strategy class
│   │   │   ├── funding_rate_arb.py # Spot long + perp short / delta-neutral
│   │   │   ├── spot_perp_basis.py # Basis trade (spot vs perp price gap)
│   │   │   └── cross_exchange.py # Cross-exchange spot price divergence
│   │   ├── scorer.py          # Fee/slippage-aware profit calculator
│   │   ├── models.py          # Opportunity Pydantic models
│   │   └── requirements.txt
│   ├── risk-engine/          # Position limits, drawdown, circuit breakers
│   │   ├── Dockerfile
│   │   ├── main.py
│   │   ├── rules/
│   │   │   ├── position_limits.py
│   │   │   ├── drawdown_guard.py
│   │   │   ├── exchange_health.py # API latency, rate limit monitoring
│   │   │   └── kill_switch.py     # Emergency halt all activity
│   │   ├── state.py           # Redis-backed risk state
│   │   ├── models.py
│   │   └── requirements.txt
│   ├── paper-trader/         # Simulate execution without real orders
│   │   ├── Dockerfile
│   │   ├── main.py
│   │   ├── simulator/
│   │   │   └── execution_model.py # Models fees, slippage, partial fills

```

```

├── funding_simulator.py # Simulate funding payment collection
├── spread_collapse.py   # Simulate spread closing unexpectedly
├── ledger.py            # Append paper trades to BigQuery
├── models.py
├── requirements.txt

├── trade-ledger/        # Audit-grade trade + event record
├── Dockerfile
├── main.py
├── schema/
├── │ trades.sql         # BigQuery table schema
├── │ opportunities.sql
├── │ funding_events.sql
├── │ └── risk_events.sql
├── writer.py           # Pub/Sub → BigQuery streaming insert
├── tax_export.py        # Generate IRS-compatible trade CSV
├── requirements.txt

├── ai-ops-agent/        # MCP server for Claude/Gemini ops agent
├── Dockerfile
├── main.py
├── mcp_server.py        # MCP protocol server
├── tools/
├── │ read_tools.py      # READ-only: balances, PnL, spread status
├── │ report_tools.py    # Generate daily/weekly reports
├── │ anomaly_tools.py   # Detect trade anomalies, slippage spikes
├── │ └── propose_tools.py # PROPOSE param changes (human approves)
├── permissions.py       # READ / PROPOSE / (never EXECUTE) model
├── prompts/
├── │ daily_report.txt
├── │ └── anomaly_alert.txt
├── requirements.txt

├── execution-orchestrator/ # Routes approved signals to Hummingbot
├── Dockerfile
├── main.py
├── hummingbot_client.py  # Hummingbot REST API client
├── approval_gate.py     # Requires human Slack approval for WRITE ops
├── position_reconciler.py # Reconcile Hummingbot vs exchange truth
├── requirements.txt

├── strategies/
├── │ hummingbot/
├── │ │ funding_rate_arb_config.yml
├── │ │ spot_perp_basis_config.yml
├── │ │ └── README.md
├── │ └── backtests/
├── │ │ funding_rate_backtest.ipynb
├── │ │ └── cross_exchange_backtest.ipynb

├── shared/
├── │ models/
├── │ │ opportunity.py
├── │ │ trade.py
├── │ │ risk_state.py
├── │ │ └── exchange_tick.py

```

```

├── pubsub/
│   ├── schemas/
│   │   ├── opportunity_detected.json
│   │   ├── trade_executed.json
│   │   └── risk_alert.json
│   └── publisher.py
├── utils/
│   ├── fee_calculator.py
│   ├── slippage_model.py
│   └── exchange_normalizer.py
├── dashboard/
│   ├── index.html                # Single-page ops dashboard
│   └── assets/
└── docs/
    ├── ARCHITECTURE.md
    ├── STRATEGY_GUIDE.md
    ├── RISK_POLICY.md
    ├── EXCHANGE_FEES.md
    └── TAX_COMPLIANCE.md

```

Phase 1 Build Instructions (Generate All Files)

1. Shared Models (shared/models/)

Generate Pydantic v2 models for all core data types:

opportunity.py:

```

from pydantic import BaseModel, Field
from enum import Enum
from datetime import datetime

class StrategyType(str, Enum):
    FUNDING_RATE_ARB = "funding_rate_arb"
    SPOT_PERP_BASIS = "spot_perp_basis"
    CROSS_EXCHANGE = "cross_exchange"
    TRIANGULAR = "triangular"

class Opportunity(BaseModel):
    id: str                                # UUID
    strategy: StrategyType
    asset: str                             # e.g., "BTC"
    long_exchange: str
    short_exchange: str
    gross_spread_bps: float
    trading_fees_bps: float
    slippage_estimate_bps: float
    funding_rate_annualized_pct: float
    net_edge_bps: float                    # gross - fees - slippage
    confidence_score: float                # 0.0 - 1.0
    recommended_size_usd: float

```

```

min_hold_hours: float
detected_at: datetime
execute: bool = False # Only True after risk engine approval
rejection_reason: str | None = None

```

trade.py:

```

from pydantic import BaseModel
from enum import Enum
from datetime import datetime

class TradeType(str, Enum):
    PAPER = "paper"
    LIVE = "live"

class TradeLeg(BaseModel):
    exchange: str
    side: str # "buy" | "sell"
    asset: str
    size: float
    fill_price: float
    fee_usd: float
    slippage_usd: float
    filled_at: datetime

class Trade(BaseModel):
    id: str
    opportunity_id: str
    type: TradeType
    legs: list[TradeLeg]
    gross_pnl_usd: float
    net_pnl_usd: float
    status: str # "open" | "closed" | "failed"
    opened_at: datetime
    closed_at: datetime | None = None
    funding_collected_usd: float = 0.0
    notes: str = ""

```

2. Opportunity Engine — Scorer (shared/utils/fee_calculator.py)

```

def calculate_net_edge(
    gross_spread_bps: float,
    long_exchange_taker_fee_bps: float,
    short_exchange_taker_fee_bps: float,
    slippage_long_bps: float,
    slippage_short_bps: float,
    funding_rate_bps_per_period: float = 0.0,
    transfer_cost_bps: float = 0.0,
    borrow_cost_bps: float = 0.0,
    safety_buffer_bps: float = 3.0,
) -> dict:
    """
    Returns net edge and full cost breakdown.

```

```

Minimum viable threshold: net_edge_bps > 8.0 AND persistence > 4 hours.
"""
total_cost_bps = (
    long_exchange_taker_fee_bps
    + short_exchange_taker_fee_bps
    + slippage_long_bps
    + slippage_short_bps
    + transfer_cost_bps
    + borrow_cost_bps
    + safety_buffer_bps
)
net_edge_bps = gross_spread_bps + funding_rate_bps_per_period - total_cost_bps
return {
    "gross_spread_bps": gross_spread_bps,
    "total_cost_bps": total_cost_bps,
    "net_edge_bps": net_edge_bps,
    "is_viable": net_edge_bps > 8.0,
    "cost_breakdown": {
        "long_fee": long_exchange_taker_fee_bps,
        "short_fee": short_exchange_taker_fee_bps,
        "slippage_long": slippage_long_bps,
        "slippage_short": slippage_short_bps,
        "transfer": transfer_cost_bps,
        "borrow": borrow_cost_bps,
        "buffer": safety_buffer_bps,
    },
}

```

3. Risk Engine Hard Rules (services/risk-engine/rules/)

Generate `kill_switch.py` with:

- `trigger_kill_switch(reason: str)` → publishes `KILL_SWITCH_ACTIVATED` event to Pub/Sub
- Sets Redis key `risk:kill_switch:active = true`
- Blocks ALL outbound trade signals
- Sends Slack alert via webhook
- Requires manual reset (`reset_kill_switch(auth_token)`) to re-enable

Generate `drawdown_guard.py` with:

```

DEFAULT_LIMITS = {
    "max_daily_loss_pct": 1.0,          # 1% of capital
    "max_per_trade_capital_pct": 2.0,   # 2% max per trade
    "max_exchange_exposure_pct": 25.0,  # Max 25% per venue
    "max_single_asset_exposure_pct": 20.0,
    "max_leverage_multiplier": 1.0,     # 1x during MVP
    "min_net_edge_bps": 8.0,
    "max_api_latency_ms": 500,
}

```

4. AI Ops Agent Permission Model (services/ai-ops-agent/permissions.py)

```
class PermissionLevel(str, Enum):
    READ = "read"          # Balances, PnL, spreads, logs – AUTONOMOUS
    PROPOSE = "propose"     # Param changes – requires Slack approval
    NEVER = "never"         # Withdrawals, leverage increase – BLOCKED ALWAYS

TOOL_PERMISSIONS = {
    # READ tools – AI executes autonomously
    "get_balances": PermissionLevel.READ,
    "get_daily_pnl": PermissionLevel.READ,
    "get_open_positions": PermissionLevel.READ,
    "get_exchange_health": PermissionLevel.READ,
    "get_opportunity_feed": PermissionLevel.READ,
    "generate_daily_report": PermissionLevel.READ,
    "detect_anomalies": PermissionLevel.READ,

    # PROPOSE tools – AI proposes, human approves via Slack
    "propose_risk_limit_change": PermissionLevel.PROPOSE,
    "propose_size_adjustment": PermissionLevel.PROPOSE,
    "propose_strategy_pause": PermissionLevel.PROPOSE,

    # NEVER – hardcoded blocked
    "withdraw_funds": PermissionLevel.NEVER,
    "increase_leverage": PermissionLevel.NEVER,
    "add_exchange_key": PermissionLevel.NEVER,
    "disable_kill_switch": PermissionLevel.NEVER,
}
```

5. Terraform Infrastructure

Generate infra/terraform/modules/cloud-run/main.tf with:

- Cloud Run service (gen2)
- Service account with least-privilege IAM
- Secret Manager secret mounting (env vars)
- Min instances: 1 (always warm)
- CPU always allocated for WebSocket services
- VPC connector for private Redis access
- Health check: /healthz endpoint

Generate infra/terraform/modules/pubsub/main.tf with these topics:

```
arb-market-data          # Raw tick data
arb-funding-rates        # Normalized funding rates
arb-opportunities        # Scored opportunity events
arb-risk-alerts          # Kill switch, drawdown alerts
arb-trade-fills          # Paper and live fill events
```

```
arb-ai-proposals      # AI-proposed parameter changes
arb-audit-log         # All system events for compliance
```

Generate `infra/terraform/modules/bigquery/main.tf` with datasets:

- `arb_market_data` — tick data, order books (90-day retention)
- `arb_trading` — opportunities, trades, funding events (7-year retention for tax)
- `arb_risk` — risk events, kill switch history
- `arb_ai_ops` — AI reports, anomaly detections, proposals

6. CCXT Market Data Collector (`services/market-data/main.py`)

Generate a production WebSocket collector using `ccxt.pro` that:

- Connects to Coinbase Advanced, Kraken, [Crypto.com](https://crypto.com), [Binance.US](https://binance.us)
- Uses `watchTicker()` and `watchOrderBook()` for low-latency data
- Reconnects automatically on disconnect (exponential backoff, max 30s)
- Publishes normalized `ExchangeTick` events to Pub/Sub `arb-market-data`
- Tracks API rate limits and pauses before hitting thresholds
- Health endpoint at `/healthz` for Cloud Run liveness check
- All API keys loaded from Secret Manager (never hardcoded)

7. Paper Trading Simulator (`services/paper-trader/simulator/execution_model.py`)

Generate a realistic execution simulator that models:

- **Fee model:** Tiered maker/taker by exchange (Coinbase: 0.4%/0.6% taker, Kraken: 0.16%/0.26% taker for base tier)
- **Slippage model:** Linear function of trade size vs order book depth (1–3 bps for \$10K, 5–15 bps for \$100K)
- **Partial fill risk:** 20% probability of <100% fill on taker orders in thin markets
- **Spread collapse:** If spread halves within 1 hour, position is exited early
- **Funding payments:** Accrue every 8 hours (CEX) or 1 hour (Hyperliquid)
- **Latency simulation:** Add 50–150ms execution delay per leg

8. Dashboard (`dashboard/index.html`)

Generate a single-page HTML/CSS/JS operations dashboard with:

- **Header:** System status indicator (green=running, yellow=degraded, red=kill switch active)
- **KPI Row:** Daily PnL | Open Positions | Active Opportunities | Exchange Health
- **Opportunity Feed:** Live table of opportunities with `net_edge_bps`, strategy, asset, pair, age

- **Funding Rate Heatmap:** Grid of assets × exchanges showing current funding rates (color-coded)
- **Paper PnL Chart:** 30-day cumulative PnL line chart (Chart.js)
- **Risk Dashboard:** Current drawdown vs limits, per-exchange exposure bars
- **AI Report Panel:** Latest AI ops agent daily summary
- **Audit Log:** Last 50 system events with timestamp
- **Dark mode default** with teal accent (#01696f)
- Refreshes via polling every 10 seconds (or WebSocket if backend provides it)

Use the Nexus design system (dark mode default for trading ops):

```
:root {
  --color-bg: #0d1117;
  --color-surface: #161b22;
  --color-border: rgba(255,255,255,0.08);
  --color-primary: #4f98a3;
  --color-text: #e6edf3;
  --color-text-muted: #8b949e;
  --color-success: #3fb950;
  --color-warning: #d29922;
  --color-error: #f85149;
}
```

9. CI/CD (.github/workflows/ci.yml)

Generate GitHub Actions CI pipeline that runs on PR:

```
# Runs on every PR to main
# Steps:
# 1. Python lint (ruff)
# 2. Type check (mypy)
# 3. Unit tests (pytest with coverage >= 80%)
# 4. Terraform validate + plan (on infra/ changes)
# 5. Docker build (smoke test each service builds)
# 6. Security scan (bandit for Python secrets)
```

10. README.md

Generate comprehensive README with:

- System architecture diagram (ASCII or Mermaid)
- 6-step quickstart (clone → secrets → infra → paper trading → validate → go live)
- Exchange setup guide (which exchanges, API key permissions required)
- Risk policy summary
- AI ops agent setup (MCP server connection to Claude Code)

- Tax compliance notes
- Contributing guide

Exchange Fee Reference (Build Into docs/EXCHANGE_FEES.md)

Exchange	Maker	Taker	Perps	Notes
Coinbase Advanced	0.0% (with promo)	0.4–0.6%	No US perps	Use Advanced Trade API
Kraken	0.16%	0.26%	Kraken Pro perps	Good for spot arb
<u>Crypto.com</u>	0.075%	0.075%	Yes	Competitive fees
<u>Binance.US</u>	0.1%	0.1%	No perps	Spot only for US
Hyperliquid	0.02%	0.05%	Yes (DEX)	Best perp fees

Security Constraints (Enforce These Everywhere)

1. Exchange API keys: Trade + Read permissions ONLY. Never withdraw permissions.
2. Secret Manager: All keys fetched at runtime, never in env files or code.
3. Kill switch: Redis key `risk:kill_switch:active` checked before every signal.
4. AI agent: Cannot call withdrawal or leverage endpoints. Enforced in `permissions.py`.
5. Audit log: Every system action emits to `arb-audit-log` Pub/Sub topic.
6. No withdrawal perms: Enforce at exchange level AND in Secret Manager IAM.

Step-by-Step Execution Order for Claude Code

When you receive this prompt, execute in this order:

1. `git init` → create root repo structure
2. Generate all `shared/` models and utilities first
3. Generate `services/market-data/` (read-only CCXT collector)
4. Generate `services/funding-rate-service/`
5. Generate `services/opportunity-engine/`
6. Generate `services/risk-engine/` (include kill switch + drawdown guard)
7. Generate `services/paper-trader/`
8. Generate `services/trade-ledger/`
9. Generate `services/ai-ops-agent/` with MCP server
10. Generate `services/execution-orchestrator/` (last — needs all other services)
11. Generate `infra/terraform/` modules
12. Generate `.github/workflows/`

13. Generate `dashboard/index.html`

14. Generate all `docs/`

15. Generate root `README.md` with full architecture diagram

Do NOT implement live execution until paper trading is validated for 14+ days.

Phase 2 (After Paper Trading Validation — Do Not Build Yet)

- Live execution via Hummingbot (funding-rate strategy, spot-perp strategy)
- Hummingbot MCP integration for AI-controlled bot management
- Solana MEV satellite (Jito bundles, Yellowstone gRPC) — separate budget
- Capital scaling automation (\$2K → \$10K → \$50K → \$100K+)

Notes on Your Stack

This system is designed to match your existing GCP/Terraform/CI-CD patterns from Agenuit/SayPadi:

- Same Cloud Run + Pub/Sub + BigQuery event-driven architecture
- Same Terraform module structure
- Same GitHub PR workflow (rebase-before-merge)
- AI ops agent uses MCP protocol — same as your existing Claude/Gemini agent integrations
- Cursor-friendly: each service is self-contained with its own `requirements.txt` and `Dockerfile`